

ЛАБОРАТОРНА РОБОТА № 3

3.1 Тема

Робота зі списками та словниками в мові програмування Python

3.2 Мета роботи

Ознайомитись із складними типами даних у мові програмування Python — списками та словниками, навчитися створювати, змінювати та обробляти їх елементи, застосовувати основні методи роботи зі структурами даних під час розв’язання прикладних задач.

3.3 Навчальні результати

Після виконання лабораторної роботи здобувач освіти повинен:

1) знати:

- призначення та особливості використання списків і словників у мові програмування Python;
- основні способи створення, доступу та модифікації елементів списків і словників;
- базові методи роботи зі списками (*append()*, *remove()*, *sort()* тощо);
- принципи зберігання даних у словниках за схемою «ключ–значення».

2) уміти:

- створювати та ініціалізувати списки й словники в Python;
- здійснювати доступ до елементів списків за індексом та до значень словника за ключем;
- додавати, змінювати та видаляти елементи у списках і словниках;
- виконувати пошук, сортування та перевірку наявності елементів у структурах даних;
- застосовувати списки та словники для розв’язання прикладних задач.

3) набути навичок:

- використання складених типів даних для зберігання та обробки інформації;
- написання програм з використанням списків і словників;

Основи розробки програмного забезпечення мовою програмування Python.

к.т.н., доц. Барбарук Віктор Миколайович

- аналізу та обробки структурованих даних у середовищі Python;
- налагодження та тестування програм, що працюють зі складними структурами даних.

3.4 Матеріально-технічне забезпечення

- персональний комп'ютер;
- операційна система Windows / Linux / macOS;
- середовище програмування (IDLE, PyCharm, VS Code або онлайн-інтерпретатор Python);
- встановлений інтерпретатор Python версії 3.x.

3.5 Теоретичні відомості

У мові програмування Python для зберігання та обробки наборів даних використовуються складені типи даних, серед яких найбільш поширеними є списки (***list***) та словники (***dict***). Вони дозволяють ефективно організовувати інформацію та виконувати над нею різноманітні операції.

Списки (***list***)

Список — це впорядкована, змінювана послідовність елементів, яка може містити дані різних типів. Кожен елемент списку має порядковий номер — індекс, що починається з нуля.

Списки застосовуються у випадках, коли необхідно:

- зберігати набір однотипних або різнотипних даних;
- обробляти елементи у визначеному порядку;
- виконувати багаторазові операції над колекцією значень (пошук, сортування, обчислення).

Приклад створення списку:

```
numbers = [10, 20, 30, 40]
```

Основні особливості списків:

- елементи можна змінювати після створення списку;
- підтримується доступ до елементів за індексом;

- дозволяється додавання та видалення елементів під час виконання програми;

- список може містити вкладені списки.

Списки часто використовуються для зберігання результатів обчислень, переліків значень, наборів чисел або текстових даних.

Словники (dict)

Словник — це змінювана структура даних, яка зберігає пари «ключ–значення». На відміну від списків, доступ до даних у словнику здійснюється не за індексом, а за унікальним ключем.

Словники доцільно використовувати, коли:

- дані мають логічний зв'язок між назвою (ключем) і значенням;
- необхідно швидко знаходити інформацію за певною ознакою;
- дані не потребують упорядкування за індексом.

Приклад створення словника:

```
student = {"name": "Олег", "age": 19, "group": "IT-25"}
```

Основні особливості словників:

- кожен ключ є унікальним;
- значення можуть бути будь-якого типу;
- структура є змінюваною;
- словники дозволяють зберігати складні та вкладені структури даних.

Словники широко використовуються для представлення реальних об'єктів (користувачів, студентів, товарів), де кожна характеристика має власне значення.

Порівняння списків і словників

Характеристика	Список (list)	Словник (dict)
Доступ до елементів	За індексом	За ключем
Порядок елементів	Збережений	Не гарантується
Тип зберігання	Послідовність	Пара «ключ–значення»
Основне призначення	Зберігання наборів значень	Зберігання структурованих даних

Списки доцільно використовувати для зберігання впорядкованих наборів даних, а словники — для представлення структурованої інформації з чітко визначеними характеристиками. Вміння правильно обирати між списками та словниками є важливою складовою ефективного програмування мовою Python.

Основні способи створення, доступу та модифікації елементів списків і словників

Створення списків у Python

Список (*list*) — це впорядкована змінювана колекція елементів, яка може містити дані різних типів.

Основні способи створення списку:

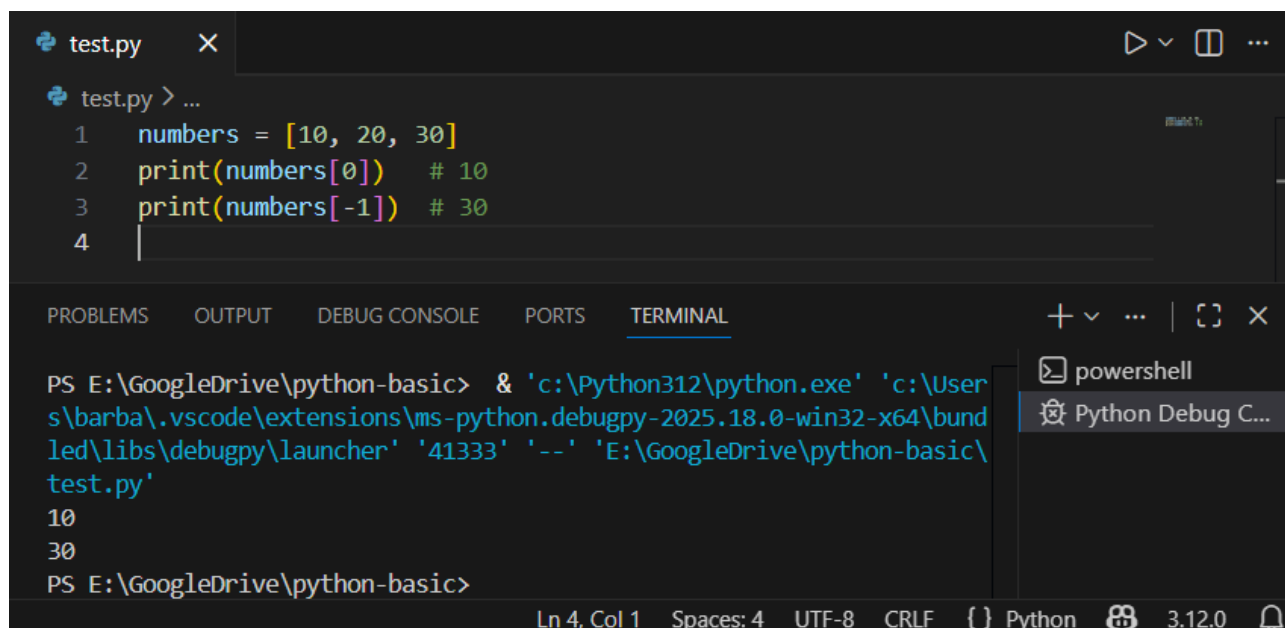
```
numbers = [1, 2, 3, 4]
mixed = [1, "text", 3.5]
empty_list = []
```

Також список можна створити за допомогою функції *list()*:

```
chars = list("Python")
```

Доступ до елементів списку

Доступ до елементів списку здійснюється за індексом, який починається з нуля.



The screenshot shows a VS Code editor with a file named `test.py`. The code in the editor is:

```
1 numbers = [10, 20, 30]
2 print(numbers[0]) # 10
3 print(numbers[-1]) # 30
4
```

Below the editor, the `TERMINAL` panel is open, showing the command prompt output:

```
PS E:\GoogleDrive\python-basic> & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '41333' '--' 'E:\GoogleDrive\python-basic\test.py'
10
30
PS E:\GoogleDrive\python-basic>
```

The status bar at the bottom indicates the cursor is at `Ln 4, Col 1`, with `Spaces: 4`, `UTF-8` encoding, `CRLF` line endings, and the file is a `Python` script.

Для доступу до частини списку використовується зріз (*slice*):

```
test.py > ...
1 numbers = [10, 20, 30]
2 print(numbers[1:3]) # [20, 30]
3

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

PS E:\GoogleDrive\python-basic> e;; cd 'e:\GoogleDrive\python-basic'
; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-p
ython.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '604
02' '--' 'E:\GoogleDrive\python-basic\test.py'
[20, 30]
PS E:\GoogleDrive\python-basic>

Ln 2, Col 32 Spaces: 4 UTF-8 CRLF { } Python 3.12.0
```

Модифікація елементів списку

Елементи списку можна змінювати безпосередньо за індексом:

```
numbers[1] = 25
```

Додавання елементів до списку:

```
numbers.append(40)      # додати в кінець
numbers.insert(1, 15)    # вставити за індексом
```

Видалення елементів зі списку:

```
numbers.remove(25)      # видалення за значенням
numbers.pop()           # видалення останнього елемента
del numbers[0]          # видалення за індексом
```

Створення словників у Python

Словник (**dict**) — це структура даних, яка зберігає пари «ключ–значення». Ключі повинні бути унікальними.

Основні способи створення словника:

```
student = {"name": "Іван", "age": 19}
empty_dict = {}
```

Альтернативний спосіб:

```
student = dict(name="Іван", age=19)
```

Доступ до елементів словника

Доступ до значень словника здійснюється за ключем:

```
print(student["name"])
```

Для безпечного доступу використовується метод **get()**:

```
print(student.get("age"))
```

Модифікація елементів словника

Зміна значення за ключем:

```
student["age"] = 20
```

Додавання нового елемента:

```
student["group"] = "ІП-21"
```

Видалення елементів словника:

```
student.pop("age")  
del student["group"]
```

Перевірка наявності елементів

Для списків:

```
if 10 in numbers:  
    print("Елемент знайдено")
```

Для словників (перевірка ключа):

```
if "name" in student:  
    print("Ключ існує")
```

Отримання наборів даних зі словника

```
keys = student.keys()  
values = student.values()  
items = student.items()
```

Ці методи дозволяють ітерувати словник у циклах.

Базові методи роботи зі списками в мові програмування Python

Список (**list**) у Python є впорядкованою змінюваною колекцією елементів. Для роботи зі списками використовується набір вбудованих методів, які дозволяють додавати, видаляти, змінювати та впорядковувати елементи.

Додавання елементів до списку

Метод ***append()*** додає новий елемент у кінець списку.

```
numbers = [1, 2, 3]
numbers.append(4)
```

Метод ***insert()*** додає елемент у список за вказаним індексом.

```
numbers.insert(1, 10)
```

Видалення елементів зі списку

Метод ***remove()*** видаляє перший елемент зі списку з указаним значенням.

```
numbers.remove(2)
```

Метод ***pop()*** видаляє елемент за індексом та повертає його значення. Якщо індекс не вказано, видаляється останній елемент.

```
last_element = numbers.pop()
```

Оператор ***del*** дозволяє видаляти елемент або частину списку за індексом.

```
del numbers[0]
```

Сортування елементів списку

Метод ***sort()*** сортує елементи списку за зростанням.

```
numbers.sort()
```

Для сортування за спаданням використовується параметр ***reverse=True***.

```
numbers.sort(reverse=True)
```

Функція ***sorted()*** повертає новий відсортований список, не змінюючи початковий.

```
new_numbers = sorted(numbers)
```

Пошук та перевірка наявності елементів

Оператор ***in*** використовується для перевірки наявності елемента у списку.

```
if 3 in numbers:
    print("Елемент знайдено")
```

Метод ***index()*** повертає індекс першого входження елемента у список.

```
position = numbers.index(3)
```

Отримання інформації про список

Метод ***len()*** повертає кількість елементів у списку.

```
length = len(numbers)
```

Метод ***count()*** підраховує кількість входжень заданого елемента.

```
count_of_twos = numbers.count(2)
```

Очищення та копіювання списку

Метод ***clear()*** видаляє всі елементи зі списку.

```
numbers.clear()
```

Метод ***copy()*** створює копію списку.

```
numbers_copy = numbers.copy()
```

Базові методи роботи зі списками в Python дозволяють ефективно керувати даними, змінювати структуру списку та виконувати операції пошуку й сортування. Використання методів `append()`, `remove()`, `sort()` та інших є основою для розв'язання практичних задач з обробки наборів даних у програмах мовою Python.

Принципи зберігання даних у словниках Python за схемою «ключ–значення»

У мові програмування Python словник (***dictionary***) є складеною структурою даних, призначеною для зберігання інформації у вигляді пар «ключ–значення». Кожен елемент словника складається з унікального ключа, який однозначно ідентифікує відповідне значення.

Ключ — це незмінний об'єкт (наприклад, рядок, число, кортеж), який використовується для доступу до значення.

Значення — це об'єкт довільного типу даних (число, рядок, список, словник тощо), пов'язаний з конкретним ключем.

Приклад створення словника:

```
student = {
    "name": "Іван",
    "age": 19,
    "group": "ІП-21"
}
```

Унікальність ключів

Однією з основних властивостей словника є унікальність ключів. У межах одного словника кожен ключ може зустрічатися лише один раз. Якщо під час додавання елемента використовується вже наявний ключ, попереднє значення буде замінено новим.

```
student["age"] = 20 # значення за ключем "age" буде оновлено
```

Типи даних ключів

Ключами словника можуть бути лише незмінні (hashable) типи даних:

- числа (int, float);
- рядки (str);
- логічні значення (bool);
- кортежі (tuple).

Змінні типи даних, такі як списки (list) або інші словники (dict), не можуть використовуватись як ключі.

Доступ до значень за ключем

Доступ до даних у словнику здійснюється не за індексом, а за ключем, що забезпечує швидкий і зручний доступ до інформації.

```
print(student["name"])
```

Для безпечного доступу до значення можна використовувати метод `get()`, який не викликає помилку у разі відсутності ключа.

```
print(student.get("email", "Немає даних"))
```

Додавання та видалення елементів

Словники дозволяють динамічно додавати нові пари «ключ–значення» та видаляти наявні елементи.

Додавання елемента:

```
student["email"] = "ivan@example.com"
```

Видалення елемента:

```
del student["group"]
```

Зберігання структурованих даних

Словники широко використовуються для зберігання структурованої інформації, де кожен ключ виконує роль імені поля, а значення — відповідних даних. Це робить словники зручними для моделювання об'єктів реального світу (користувачів, товарів, записів баз даних).

Приклад вкладки структури:

```
students = {  
    "student1": {"name": "Іван", "age": 19},  
    "student2": {"name": "Олена", "age": 20}  
}
```

Переваги використання словників

- швидкий доступ до даних за ключем;
- наочна та логічна організація інформації;
- підтримка вкладених структур;
- зручність для зберігання та обробки даних у програмуванні.

3.6 Порядок виконання роботи

- 1) Запустити середовище програмування Python.
- 2) Створити новий файл програми.
- 3) Ознайомитися з теоретичними відомостями до роботи.
- 4) Реалізувати загальне завдання лабораторної роботи.
- 5) Виконати індивідуальне завдання відповідно до варіанта.
- 6) Перевірити коректність роботи програми.
- 7) Оформити звіт та подати його на перевірку.

3.7 Загальне завдання

Написати програму, яка:

- вводить початкові дані списку/словника;
- обчислює результат;
- виводить результат.

3.8 Індивідуальні завдання

- 1) Написати програму для пошуку максимального елемента в списку.
- 2) Реалізувати програму для підрахунку кількості елементів у словнику.
- 3) Створити програму для видалення елемента з певним значенням зі списку.
- 4) Написати програму, яка виводить всі елементи списку у зворотному порядку.
- 5) Реалізувати програму для пошуку найменшого значення серед елементів словника.
- 6) Створити програму для перевірки наявності заданого значення у словнику.
- 7) Написати програму для сортування списку за зростанням.
- 8) Реалізувати програму для перевірки, чи є два списки однаковими.
- 9) Написати програму для об'єднання двох списків в один.
- 10) Реалізувати програму для видалення всіх повторюваних елементів зі списку.

Нульовий варіант (зразок виконання)

Створити програму мовою Python, яка:

- створює список цілих чисел;
- створює словник, що містить імена людей та їхній вік;
- знаходить і виводить максимальний елемент списку;
- обчислює середній вік осіб, вік яких перевищує задане значення.

Програмний код (приклад реалізації)

```
# Створення списку цілих чисел
numbers = [12, 5, 8, 20, 3, 15]

# Створення словника з іменами та віком
people = {
    "Олег": 18,
    "Анна": 21,
    "Марія": 19,
    "Іван": 23,
    "Петро": 17
}
```

```

# Задання граничного значення віку
age_limit = 18

# Пошук максимального елемента у списку
max_number = numbers[0]
for number in numbers:
    if number > max_number:
        max_number = number

# Виведення максимального значення
print("Максимальний елемент списку:", max_number)

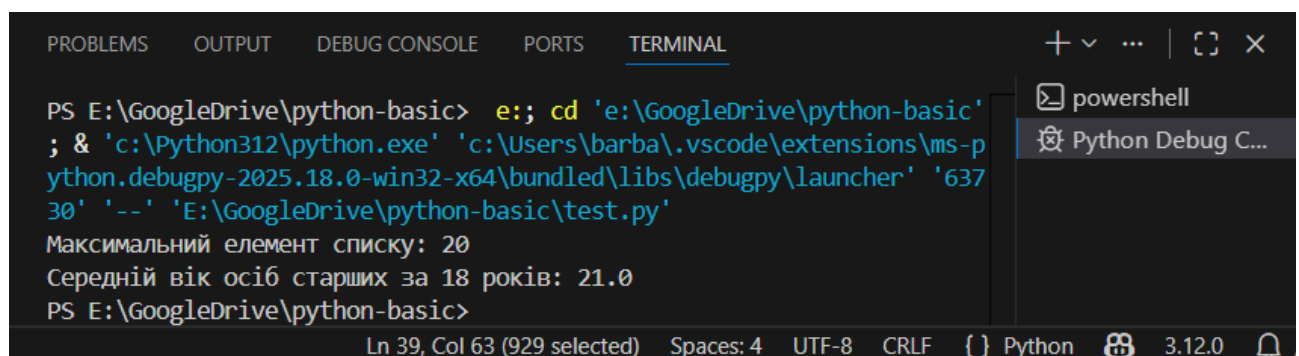
# Обчислення середнього віку людей, старших за заданий вік
sum_age = 0
count = 0

for age in people.values():
    if age > age_limit:
        sum_age += age
        count += 1

# Перевірка, щоб уникнути ділення на нуль
if count > 0:
    average_age = sum_age / count
    print("Середній вік осіб старших за", age_limit, "років:", average_age)
else:
    print("Осіб, старших за", age_limit, "років, не знайдено")

```

Результат виконання:



```

PS E:\GoogleDrive\python-basic> e;; cd 'e:\GoogleDrive\python-basic'
; & 'c:\Python312\python.exe' 'c:\Users\barba\.vscode\extensions\ms-p
ython.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '637
30' '--' 'E:\GoogleDrive\python-basic\test.py'
Максимальний елемент списку: 20
Середній вік осіб старших за 18 років: 21.0
PS E:\GoogleDrive\python-basic>

```

Пояснення до програми:

- список `numbers` використовується для зберігання числових даних;
- словник `people` містить пари «ім'я – вік»;
- пошук максимального елемента реалізовано за допомогою циклу `for`;

- обчислення середнього віку виконується лише для осіб, вік яких перевищує задане значення;
- додаткова перевірка запобігає помилці ділення на нуль.

3.9 Вимоги до звіту

Звіт з лабораторної роботи повинен містити:

- 1) Назву лабораторної роботи.
- 2) Мету роботи.
- 3) Короткі теоретичні відомості.
- 4) Умову загального та індивідуального завдання.
- 5) Текст програми з коментарями.
- 6) Результати виконання програми (скріншоти або приклади виводу).
- 7) Висновок.

3.10 Контрольні питання

- 1) Як створити список у Python?
- 2) Яким чином додати новий елемент до списку?
- 3) Як створити словник у Python?
- 4) Як перевірити наявність ключа у словнику?
- 5) Які основні методи використовуються для роботи зі списками?
- 6) Чим відрізняється список від словника?
- 7) Як отримати всі ключі та значення словника?
- 8) Як відсортувати список у Python?

3.11 Літературні джерела

- 1) Python Software Foundation. – URL: <http://www.python.org>

2) Яковенко А. В. Основи програмування. Python. Частина 1 [Електронний ресурс]: підручник для студентів спеціальності 122 «Комп'ютерні науки». Київ: КПІ ім. Ігоря Сікорського, 2018. 195 с.
<https://ela.kpi.ua/bitstream/123456789/25111/1/Python.pdf>